

MYZ 307 - PROJE FİNAL RAPORU



Dersten Sorumlu Akademisyen: Prof. Dr. Bilge Günsel

Ata Ataş - 040230771

Alim Devrim Erol - 040210544

İhsan Arınç Dokucu - 040200398

Berat Gökay TOPCU - 040210118

24 Aralık 2025

Contents

1	Konu Başlığı, Referans Bildiri ve GitHub Sayfası	4
2	Problemin Tanımı	4
2.1	Giriş ve Çıkışlar	4
2.2	Blok Şema Açıklaması	6
2.3	Encoder Çıktıları $e_1, \dots, e_5$ ve Çözünürlüklerin Azaltılması	8
2.4	Decoder Yapısı ve Düşük Çözünürlükten Yüksek Çözünürlüğe Geçiş	8
3	Segmentasyon Süreci ve Model Mimarisi	9
3.1	Conv-BN-ReLU Blokları	10
3.2	Conv-BN-ReLU Bloklarının Matematiksel Tanımı	10
3.3	Çözünürlük Azaltma ve Özellik (Feature) Artırma Mekanizması	11
3.4	ResNet-34 Encoder Backbone Mimarisi	11
3.5	U-Net Benzeri Segmentasyon Decoder	15
4	Kayıp Fonksiyonları ve Öğrenme Tekniği	15
4.1	Toplam Kayıp Fonksiyonu ($\mathcal{L}_{total}$)	15
4.2	Kayıp Fonksiyonlarının Matematiksel Modellenmesi	16
4.2.1	Segmentasyon Kaybı ($\mathcal{L}_{seg}$)	16
4.2.2	Inpainting Kaybı ($\mathcal{L}_{inp}$)	16
4.2.3	Gradyan Akışı ve Optimizasyon Mekanizması	16
4.3	Optimizasyon ve Ağırlık Güncelleme Mekanizması	17
4.4	Gradyan	17
4.4.1	Zincir Kuralı (Chain Rule) ve w Ağırlıklarına Ulaşım	17
4.5	Değiştirilen Ağırlıkların Sisteme Etkisi	17
5	Inpainting Süreci ve Gated Feature Fusion (GFF) Mekanizması	17
5.1	Kaba Segmentasyona Dayalı Maskeleme	18
5.2	Inpainting için Encoder Özelliklerinin Çıkarılması	18
5.3	Gated Feature Fusion (GFF) Mekanizması	19
5.4	Inpainting Decoder ile Boşluk Doldurma	20
5.5	Coarse-to-Fine Segmentasyon ile Nihai Maske Üretimi	21
5.6	Öğrenme Mantiğı ve Düzeltme (Correction Learning)	22
6	DeneySEL Sonuçlar ve Detaylı Analiz	22
6.1	Test Kümesi Performansı (Nicel Analiz)	23
6.2	Kvasir-SEG Test Kümesi Üzerinde Elde Edilen Performans Sonuçları	23
6.3	Tartışma	24
6.4	Transfer Öğrenme ve Genelleştirme Analizi (ISBI 2016)	25
6.5	Deney Kurgusu ve Hiperparametreler	25
6.6	Train Sonuçları	25

1 Konu Başlığı, Referans Bildiri ve GitHub Sayfası

Bu proje kapsamında, “Self-Supervised Correction Learning for Semi-Supervised Biomedical Image Segmentation” başlıklı çalışmada¹ önerilen *Self-Supervised Correction Learning (SemiMedSeg)* yaklaşımı, kolonoskopi görüntülerinde polip segmentasyonu problemi için incelenmekte ve yeniden uygulanmaktadır.

- **Konu Başlığı:** Yarı Denetimli Biyomedikal Görüntü Bölütleme için Kendinden Denetimli Düzeltme Öğrenmesi (Self-Supervised Correction Learning).
- **Referans Bildiri:**
 - R. Zhang, S. Liu, Y. Yu, G. Li, “Self-Supervised Correction Learning for Semi-supervised Biomedical Image Segmentation”, *MICCAI 2021*.
- **GitHub Deposu:**
 - <https://github.com/ReaFly/SemiMedSeg>

Bu çalışmada, referans kodun `polyp` deney ayarları kullanılarak Kvasir-SEG veri kümesi üzerinde eğitimi gerçekleştirilmekte ve özellikle maskeleme–inpainting süreci ile yarı denetimli öğrenme mantığı analiz edilmektedir.²

2 Problemin Tanımı

Problem, girişte bir kolonoskopi görüntüsü (polip içerebilen) verilen bir sistemde, çıktı olarak bu görüntüdeki polip bölgesinin piksel bazında segmentasyon maske haritasının üretilmesidir. Sistem ayrıca, segmentasyon sonucunda elde edilen polip alanını maskeleyip, maske ile gizlenen bölgeyi *inpainting* ile yeniden oluşturmaya çalışan ikinci bir görev (inpainting görevi) içermektedir.

2.1 Giriş ve Çıkışlar

- **Giriş:**
 - Endoskopi görüntüleri matematiksel olarak

$$I \in \mathbb{R}^{H \times W \times 3}$$

şeklinde ifade edilebilir. Burada:

- * H (*height*): Görüntünün piksel cinsinden yüksekliği,
- * W (*width*): Görüntünün piksel cinsinden genişliği,
- * 3: Renk kanallarının sayısıdır (RGB: kırmızı, yeşil, mavi).

Dolayısıyla her piksel üç renk kanalından oluşan bir vektörle temsil edilmektedir.

- Renkli endoskopi görüntüsü $I \in \mathbb{R}^{H \times W \times 3}$.

¹R. Zhang *et al.*, MICCAI 2021.

²Kvasir-SEG veri kümesi bağırsak polip segmentasyonu için standart bir benchmark veri kümesidir.

– *Ground Truth* Maskesi

$$Y \in \{0, 1\}^{H \times W}$$

şeklinde tanımlanır. Burada:

- * $Y(i, j) = 1$ ise (i, j) pikseli polip bölgesine aittir,
- * $Y(i, j) = 0$ ise (i, j) pikseli arka plana (healthy doku) aittir.

Bu maske, uzman anotasyonları ile elde edilir ve modelin çıkışı $\hat{Y}$ ile karşılaştırılarak öğrenme süreci denetlenir.

– Yalnızca etiketli veri için: Piksel bazlı *ground truth* segmentasyon maskesi $Y \in \{0, 1\}^{H \times W}$.

Maskeleme süreci şu şekilde tanımlanabilir:

$$I_{\text{masked}} = I \cdot (1 - Y),$$

yani polip alanına denk gelen pikseller sıfırlanarak gizlenir. Inpainting decoder, bu eksik bölgeleri çevre bağlamı kullanarak yeniden doldurmayı öğrenir. Böylece model:

- * Polip bölgesinin *nerede* olduğunu (segmentasyon aracılığıyla),
- * Polip bölgesinin *nasıl bir yapıya sahip olduğunu* (inpainting ile)

öğrenmiş olur.

• Çıkışlar:

- Polip segmentasyon çıktısı (olasılık haritası) $\hat{Y}_{coarse} \in [0, 1]^{H \times W}$.
- Inpainting çıktısı: Polip alanlarının maskelenmesi ile oluşturulan bozuk görüntünün, ağ tarafından rekonstrüksiyonu $\hat{I}_{\text{inp}}$.

İncelenen makaleden süreç anlatımı yapan görsel alıntısı aşağıda verilmiştir:

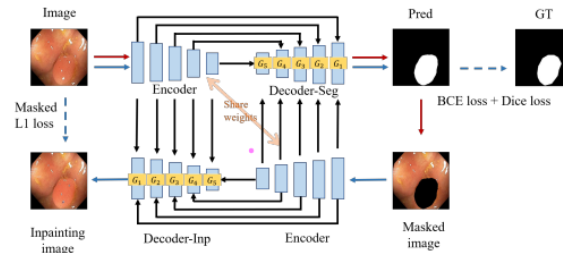
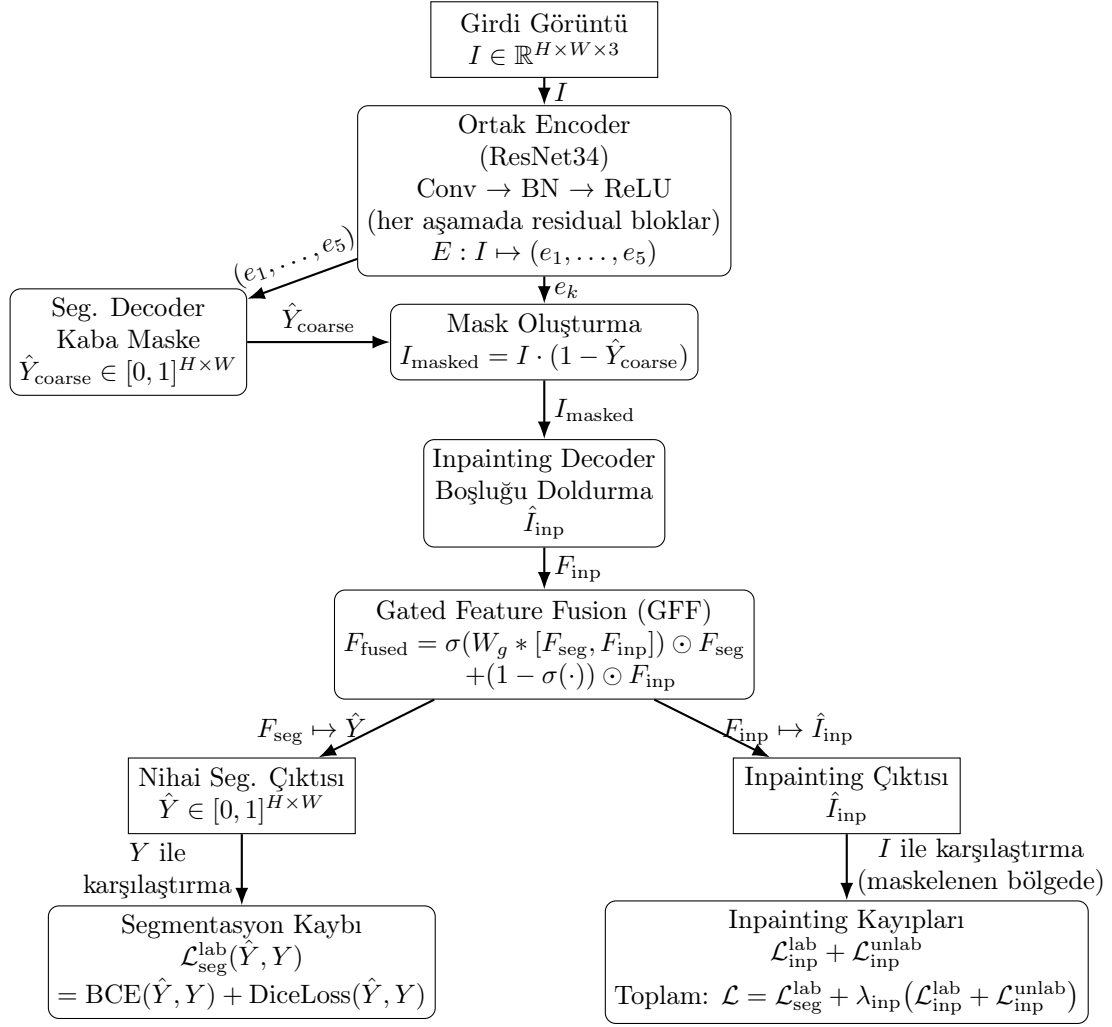


Fig. 1. The overview of our network. Both encoders share weights. G_1 - G_5 represent five GFF modules. The red and blue arrows denote the input and output of our network in the first and second stage respectively.

Figure 1: Süreç Diyagramı

Aşağıdaki diyagram, kodun yüksek seviyede nasıl çalıştığını ve masking–inpainting sürecinin akışını göstermektedir.



2.2 Blok Şema Açıklaması

Yüksek seviyede sistem şu bloklardan oluşmaktadır:

1. Ortak bir **Encoder** bloğu (ResNet34 tabanlı öznetelik çıkarıcı):

- 34 katmanlı bir evrişimsel sinir ağıdır,
- Evrişimsel Sinir Ağları (CNN), özellikle görüntü verilerindeki uzamsal (spatial) ilişkileri öğrenmek için tasarlanmış derin öğrenme mimarileridir. CNN’lerin temel yapı taşı olan *evrişim katmanı*, küçük boyutlu filtrelerin (kernel) görüntü üzerinde kaydırılmasıyla yani konvülsyon işlemi yapılmasıyla özellik haritaları üretir. Bu işlem matematiksel olarak şu şekilde tanımlanır:

$$(f * g)(i, j) = \sum_m \sum_n f(m, n) g(i - m, j - n),$$

burada f , görüntüyü veya ara özellik haritasını; g ise öğrenilebilir filtreyi ifade eder. Bu yapı, kenar, doku, renk geçişi ve şekil gibi yerel özelliklerin otomatik olarak çıkarılmasını sağlar.

CNN’ler; evrişim katmanları, aktivasyon fonksiyonları (örneğin ReLU), havuzlama (pooling) katmanları ve isteğe bağlı olarak tam bağlantılı (fully connected) katmanların birleşiminden oluşur. Bu çok katmanlı yapı sayesinde model, düşük seviyeli kenarlardan yüksek seviyeli semantik yapılara kadar hiyerarşik bir özellik temsili öğrenir. CNN’lerin ağırlık paylaşımı ve yerellik

ilkesi, parametre sayısını azaltarak görüntü işleme görevlerinde yüksek verimlilik ve doğruluk sağlar.

- *residual connection* (artık bağlantı) kullanarak gradyan kaybolması problemini azaltır,
 - Derin sinir ağlarında, katman sayısı arttıkça *gradyan kaybolması* (vanishing gradient) adı verilen problem ortaya çıkabilir. Geri yayılım (backpropagation) sırasında türevlerin ardışık katmanlardan geçerken küçülmesi, özellikle erken katmanların öğrenememesine yol açar. Residual bağlantılar bu problemi azaltmak için tasarlanmıştır.

Klasik bir derin blok şu formdadır:

$$z_{l+1} = F(z_l),$$

burada F , konvolüsyon, aktivasyon ve normalizasyon işlemlerinden oluşan dönüşümü ifade eder. Residual bağlantı eklenmiş bir blok ise şu şekilde tanımlanır:

$$z_{l+1} = F(z_l) + z_l.$$

Bu yapı, bloğun girdisini doğrudan çıktıya ekleyerek bir *kimlik haritalama* (identity mapping) yolu oluşturur. Böylece gradyan, geri yayılım sırasında doğrudan şu şekilde akar:

$$\frac{\partial \mathcal{L}}{\partial z_l} = \frac{\partial \mathcal{L}}{\partial z_{l+1}} \left(1 + \frac{\partial F}{\partial z_l} \right),$$

buradaki “1” terimi, gradyanın tamamen kaybolmasını matematiksel olarak engeller. Bu sayede çok daha derin ağların stabil biçimde eğitilmesi mümkün olur. Residual bağlantılar, SemiMedSeg modelinde kullanılan ResNet34 encoder mimarisinin temel bileşenidir ve modelin hem segmentasyon hem inpainting görevlerinden gelen gradyanları etkin biçimde öğrenmesini sağlamaktadır.

- Alt katmanlarda kenar/doku, üst katmanlarda ise semantik özellikler çıkarır.

Bu encoder’dan elde edilen çok ölçekli özellik haritaları hem segmentasyon hem de inpainting decoder kollarına beslenir.

2. **Segmentasyon Decoders** (seg-Decoder): Çok ölçekli özellikleri kullanarak polip maske tahmini üretir.
3. **Inpainting Decoders** (inp-Decoder): Maskelenmiş görüntü üzerinde kayıp polip bölgesini yeniden oluşturmaya çalışır.
4. **Gated Feature Fusion (GFF)** modülleri: Segmentasyon ve inpainting dallarından gelen özellikleri kapılayarak birleştirir.
5. **Kayıp Fonksiyonları**: Etiketli veri için segmentasyon kaybı; etiketli *ve* etiketlenmemiş veri için inpainting kaybı.

Tüm bu mimari raporun ilerleyen kısımlarında detaylı olarak açıklanacaktır.

2.3 Encoder Çıktıları $e_1, \dots, e_5$ ve Çözünürlüklerin Azaltılması

SemiMedSeg modelinde giriş görüntüsü $x \in \mathbb{R}^{H \times W \times 3}$, ResNet34 tabanlı ortak bir encoder $E(\cdot)$ üzerinden beş farklı ölçekte öznitelik haritasına dönüştürülür. Bu işlem aşağıdaki gibi özetlenebilir:

$$(e_1, e_2, e_3, e_4, e_5) = E(x).$$

ResNet34 mimarisi; ilk konvolüsyon ve havuzlama katmanının ardından dört adet *residual blok grubu* içerir. Her grup, uzamsal çözünürlüğü iki kat azaltırken kanal sayısını artırır. Bu sayede giderek daha soyut fakat daha geniş alana bakan (daha büyük reseptif alanlı) öznitelikler elde edilir. Tipik bir yapı şu şekildedir:

- $e_1 \in \mathbb{R}^{64 \times \frac{H}{4} \times \frac{W}{4}}$: İlk konvolüsyon ve maksimum havuzlama katmanlarından sonra elde edilen, kenar ve temel doku bilgilerini içeren düşük seviyeli özellikler,
- $e_2 \in \mathbb{R}^{64 \times \frac{H}{4} \times \frac{W}{4}}$: Birinci residual aşama (layer1) çıktısı; çözünürlük korunurken özellikler daha ayrıntılı biçimde rafine edilir,
- $e_3 \in \mathbb{R}^{128 \times \frac{H}{8} \times \frac{W}{8}}$: İkinci residual aşama (layer2) çıktısı; daha geniş alıcı alana sahip orta seviyeli semantik özellikler,
- $e_4 \in \mathbb{R}^{256 \times \frac{H}{16} \times \frac{W}{16}}$: Üçüncü residual aşama (layer3) çıktısı; nesne seviyesinde soyut temsiller,
- $e_5 \in \mathbb{R}^{512 \times \frac{H}{32} \times \frac{W}{32}}$: Dördüncü residual aşama (layer4) çıktısı; en düşük çözünürlükte en yüksek seviyeli semantik özellikler.

Burada her residual blok, girdisini z_l ve blok içi dönüşümü $F(\cdot)$ olmak üzere

$$z_{l+1} = F(z_l) + z_l$$

şeklinde tanımlanan artık bağlantı (residual connection) ile işler. Böylece derin encoder boyunca gradyan kaybolması problemi azaltılır ve $e_1, \dots, e_5$ özniteliklerinin tümü kararlı bir şekilde öğrenilebilir.

Modelin `mymodel.py` dosyasında bu işlem aşağıdaki satırda gerçekleştirilir:

```
1 # x: [B, 3, H, W]
2 e1, e2, e3, e4, e5 = self.encoder(x)
```

Bu satır, encoder'ın giriş görüntüsünden farklı çözünürlük ve kanal sayılarında beş adet öznitelik haritası ürettiğini göstermektedir.

2.4 Decoder Yapısı ve Düşük Çözünürlükten Yüksek Çözünürlüğe Geçiş

Encoder'dan elde edilen çok ölçekli öznitelikler e_1, e_2, e_3, e_4, e_5 kullanılarak segmentasyon decoder'ı uzamsal çözünürlüğü kademeli olarak artıran bir yapı izler. Decoder, en düşük çözünürlükten başlayarak yüksek çözünürlüğe doğru ilerler.

Decoder aşamaları matematiksel olarak aşağıdaki şekilde tanımlanabilir:

$$d_5 = D_5(e_5), \quad d_i = D_i(\text{cat}(\text{Up}(d_{i+1}), e_i)), \quad i = 1, \dots, 4,$$

burada D_i ilgili DecoderBlock'u, $Up(\cdot)$ uzamsal çözünürlüğü iki katına çıkaran üst örnekleme işlemini ve $cat(\cdot)$ kanal boyutunda birleştirmeyi ifade eder.

Her DecoderBlock, üst örnekleme işlemini takiben encoder'dan gelen skip connection ile birleştirme ve ardından bir veya daha fazla Conv-BN-ReLU katmanından oluşur. Bu yapı sayesinde, encoder'dan gelen yüksek çözünürlüklü uzamsal detaylar ile decoder'daki yüksek seviyeli semantik bilgiler etkin biçimde birleştirilir.

Bu süreç sonucunda uzamsal çözünürlük aşağıdaki şekilde tersine çevrilir:

$$\frac{H}{32} \rightarrow \frac{H}{16} \rightarrow \frac{H}{8} \rightarrow \frac{H}{4} \rightarrow \frac{H}{2} \rightarrow H.$$

Modelin **forward** fonksiyonunda bu yapı PyTorch kodu ile şu şekilde uygulanır:

```

1 # Encoder'dan gelen   znitelikler
2 e1, e2, e3, e4, e5 = self.encoder(x)
3
4 # Segmentation decoder (a1 ak          znrlkten          y kse e)
5 d5 = self.segDecoder5(e5)                # H/32 -> H/16
6 d4 = self.segDecoder4(torch.cat([d5, e4], dim=1)) # H/16 -> H/8
7 d3 = self.segDecoder3(torch.cat([d4, e3], dim=1)) # H/8 -> H/4
8 d2 = self.segDecoder2(torch.cat([d3, e2], dim=1)) # H/4 -> H/2
9 d1 = self.segDecoder1(torch.cat([d2, e1], dim=1)) # H/2 -> H
10
11 # Nihai segmentasyon      kts
12 mask = self.segconv(d1)          # kanal indirgeme
13 mask = torch.sigmoid(mask)       # [0,1] aral

```

Bu yapı sayesinde:

- e_5 gibi çok düşük çözünürlüklü, yüksek seviyeli özellikler, önce üst örnekleme ile büyütülür,
- Ardından aynı seviyedeki encoder çıktısı ($e_4, e_3, \dots$) ile birleştirilerek hem *global* bağlam (derin seviyelerden) hem de *lokal* detay (yüzeye yakın encoder seviyelerinden) korunur,
- Son katmanda d_1 tam giriş çözünürlüğüne ulaşır ve tek kanallı bir maske üretilerek segmentasyon tamamlanır.

Bu süreç inpainting decoder'ı için de benzer biçimde işler; fark olarak inpainting kolu RGB uzayında bir çıktı (maskelenmiş bölgenin doldurulmuş versiyonu) üretir ve GFF modülleri, her iki decoder'dan gelen çok ölçekli öznitelikleri kapılayarak (gating) birleştirir.

3 Segmentasyon Süreci ve Model Mimarisi

SemiMedSeg modeli, çok görevli (multi-task) bir encoder-decoder yapısını temel alan, U-Net türevi(+ResNET 34) hybrid bir mimaridir. Model üç ana bileşenden oluşur: (i) ResNet34 tabanlı ortak bir encoder, (ii) segmentasyon için kullanılan bir decoder, (iii) maskeleme sonrası görüntünün yeniden oluşturulmasını sağlayan bir inpainting decoder. Bu iki decoder arasındaki bilgi paylaşımı *Gated Feature Fusion (GFF)* modülleri (bkz. Bölüm 5) ile sağlanır. Aşağıda bu bileşenler matematiksel ve yapısal açıdan detaylandırılmaktadır.

3.1 Conv–BN–ReLU Blokları

Modelin tüm encoder ve decoder katmanlarının temel yapı taşı *Convolution–Batch Normalization–ReLU* üçlüsüdür. Bu yapı şu sırayla uygulanır:

$$x \xrightarrow{\text{Conv}} z \xrightarrow{\text{BN}} \tilde{z} \xrightarrow{\text{ReLU}} f = \max(0, \tilde{z}).$$

3.2 Conv–BN–ReLU Bloklarının Matematiksel Tanımı

Modelde kullanılan tüm encoder ve decoder katmanlarının temel yapı taşı, ardışık olarak uygulanan Convolution, Batch Normalization ve ReLU aktivasyonundan oluşan Conv–BN–ReLU bloğudur. Bu blok, uzamsal özellik çıkarımı, istatistiksel normalizasyon ve doğrusal olmayanlık kazandırma işlemlerini tek bir yapı içerisinde birleştirir.

Convolution (Conv). Girdi tensörü $x \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ üzerine uygulanan bir konvolüsyon işlemi, C_{out} adet filtre kullanarak

$$y \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$$

boyutunda bir çıktı üretir. Çok kanallı konvolüsyon işlemi aşağıdaki şekilde tanımlanabilir:

$$y_{k,i,j} = \sum_{c=1}^{C_{\text{in}}} \sum_{u,v} w_{k,c,u,v} x_{c,i+u,j+v} + b_k,$$

burada $w_{k,c,u,v}$ k -ıncı filtrenin ağırlıklarını, b_k ise bias terimini ifade etmektedir. Bu işlem, yerel uzamsal örüntülerin ve kanal içi ilişkilerin öğrenilmesini sağlar.

Batch Normalization (BN). Konvolüsyon çıktıları, mini-batch istatistikleri kullanılarak normalize edilir. Bir aktivasyon kanalı için Batch Normalization işlemi şu şekilde tanımlanır:

$$\hat{y} = \frac{y - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad z = \gamma \hat{y} + \beta,$$

burada μ_B ve σ_B^2 sırasıyla mini-batch ortalaması ve varyansı, γ ve β ise öğrenilebilir ölçekleme ve kaydırma parametrelerini göstermektedir. Batch Normalization, içsel kovaryans kaymasını azaltarak gradyan akışını stabilize eder ve derin ağların daha etkin şekilde eğitilmesini mümkün kılar.

ReLU Aktivasyonu. Normalize edilmiş aktivasyonlara doğrusal olmayanlık kazandırmak amacıyla Rectified Linear Unit (ReLU) fonksiyonu uygulanır:

$$\text{ReLU}(z) = \max(0, z).$$

ReLU fonksiyonu, pozitif bölgede türevinin 1 olması sayesinde gradyan kaybolması problemini önemli ölçüde azaltır ve seyrek (sparse) aktivasyonlar oluşturarak öğrenmeyi hızlandırır.

Blok Olarak Conv–BN–ReLU. Bu üç işlem bir arada değerlendirildiğinde, Conv–BN–ReLU bloğu aşağıdaki bileşik dönüşümü temsil eder:

$$f(x) = \text{ReLU}(\text{BN}(\text{Conv}(x))).$$

Bu yapı, hem encoder hem de decoder boyunca tekrar edilerek çok ölçekli, doğrusal olmayan ve istatistiksel olarak kararlı özellik temsillerinin öğrenilmesini sağlar.

3.3 Çözünürlük Azaltma ve Özellik (Feature) Artırma Mekanizması

Evrişimli sinir ağlarında uzamsal çözünürlüğün kademeli olarak düşürülmesi ve buna karşılık özellik (feature) sayısının artırılması, modelin hem yerel detayları hem de yüksek seviyeli semantik bilgileri etkili bir biçimde öğrenebilmesi açısından temel bir tasarım ilkesidir. Bu çalışmada kullanılan mimaride bu dönüşüm, esas olarak konvolüsyon katmanlarının parametreleri aracılığıyla gerçekleştirilmektedir.

Çözünürlük Azaltma (Downsampling). Uzamsal çözünürlüğün düşürülmesi, Batch Normalization veya ReLU gibi aktivasyon tabanlı işlemlerle değil, doğrudan uzamsal örnekleme yapan katmanlarla sağlanır. Bu bağlamda, en yaygın ve etkin yöntem, stride değeri 1’den büyük olan konvolüsyonlardır. Stride değeri $s > 1$ olan bir konvolüsyon işlemi sonucunda çıktı çözünürlüğü yaklaşık olarak

$$H_{\text{out}} = \frac{H_{\text{in}}}{s}, \quad W_{\text{out}} = \frac{W_{\text{in}}}{s}$$

şeklinde azalır. ResNet-34 mimarisinde, her yeni artık aşamanın (stage) ilk bloğunda kullanılan stride’lı konvolüsyonlar sayesinde çözünürlük kademeli olarak $H/4 \rightarrow H/8 \rightarrow H/16 \rightarrow H/32$ seviyelerine indirilmektedir.

Özellik (Feature) Artırma. Özellik sayısının artırılması, konvolüsyon katmanlarında kullanılan filtre (kernel) sayısının artırılması yoluyla gerçekleştirilir. Bir konvolüsyon katmanında giriş ve çıkış kanal sayıları arasındaki ilişki

$$\text{Conv} : C_{\text{in}} \rightarrow C_{\text{out}}$$

şeklinde ifade edilir. Burada C_{out} değeri, elde edilen özellik haritası sayısını belirlemekte olup, daha yüksek kanal sayıları daha zengin ve soyut temsillerin öğrenilmesine olanak sağlar. ResNet-34 encoder yapısında kanal sayısı her aşamada $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ olacak şekilde artırılmaktadır.

Batch Normalization ve ReLU’nun Rolü. Batch Normalization ve ReLU katmanları, ne uzamsal çözünürlüğü değiştirmekte ne de kanal sayısını artırmaktadır. Bu katmanların temel işlevi, öğrenme sürecini stabilize etmek, dağılım kaymalarını azaltmak ve modele doğrusal olmayanlık kazandırmaktır. Dolayısıyla, çözünürlük azaltma ve özellik artırma işlemleri doğrudan konvolüsyon katmanları tarafından gerçekleştirilirken, Batch Normalization ve ReLU bu dönüşümlerin etkin ve kararlı bir şekilde öğrenilmesini sağlamaktadır.

Özet. Sonuç olarak, mimaride gözlenen çözünürlük düşürme ve özellik artırma mekanizması, konvolüsyon katmanlarının stride ve filtre sayısı gibi yapısal parametrelerine dayanmaktadır. Batch Normalization ve ReLU katmanları ise bu dönüşümü destekleyici bir rol üstlenerek derin ağların etkin biçimde eğitilmesine katkı sağlamaktadır.

3.4 ResNet-34 Encoder Backbone Mimarisi

Bu çalışmada, özellik çıkarımı (feature extraction) amacıyla encoder omurgası olarak ResNet-34 mimarisi kullanılmıştır. ResNet-34, derin artık (residual) öğrenme yaklaşımına dayanan ve özellikle orta derinlikte ağlarda

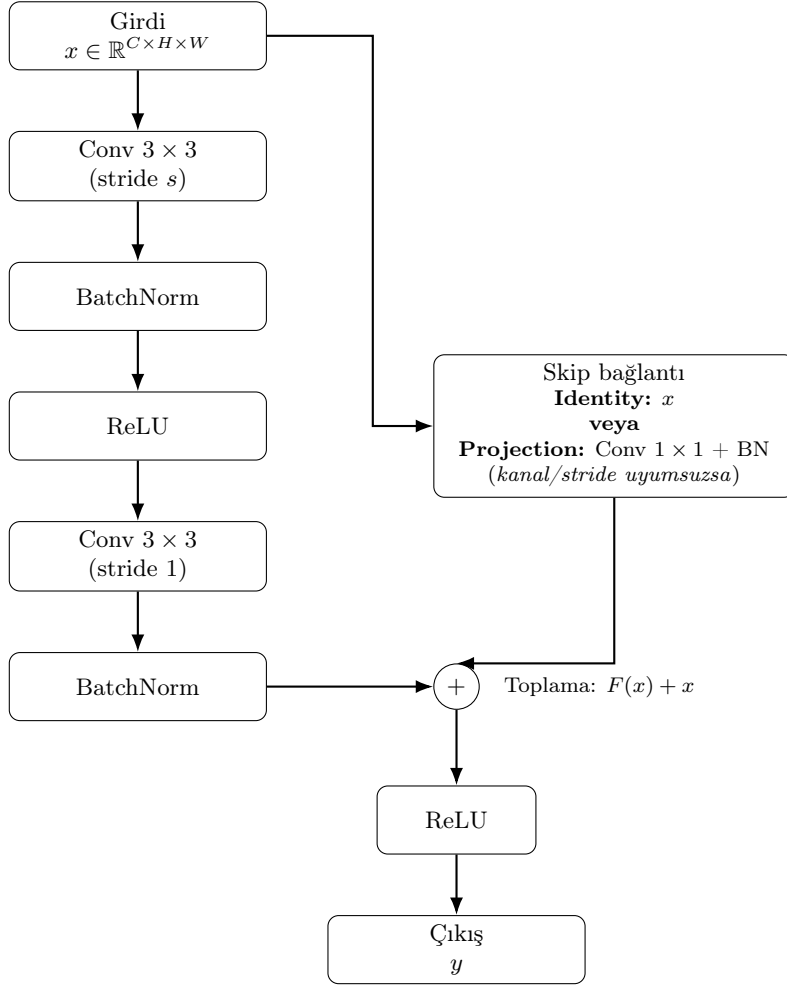


Figure 2: ResNet-34 mimarisinde kullanılan *BasicBlock* yapısı. Ana yol iki adet 3×3 Conv–BN katmanından oluşur; giriş x , kimlik (identity) veya boyut uyumsuzluğu durumunda projeksiyon (Conv 1×1 + BN) üzerinden taşınarak ana yol çıktısı ile toplanır ve ardından ReLU uygulanır.

etkin performans sunan bir evrişimli sinir ağıdır. Daha derin ResNet türevlerinden farklı olarak ResNet-34 mimarisi, *Bottleneck* bloklar yerine *BasicBlock* yapısını kullanmaktadır.

ResNet-34 mimarisinde artık blokların katmanlara dağılımı

$$[3, 4, 6, 3]$$

şeklinde ve toplamda 16 adet *BasicBlock* içermektedir.

Stem (Giriş Bloğu). Encoder yapısı, giriş görüntüsünün uzamsal çözünürlüğünü hızlı bir şekilde düşürmek ve kanal derinliğini artırmak amacıyla bir başlangıç bloğu (stem) ile başlar. Bu blok aşağıdaki bileşenlerden oluşmaktadır:

- 7×7 boyutunda, 64 çıkış kanallı, stride 2 ve padding 3 olan evrişim katmanı,
- Batch Normalization (BN),
- ReLU aktivasyon fonksiyonu,
- Stride 2 olan 3×3 maksimum havuzlama (MaxPooling) katmanı.

Stem bloğu sonrasında, giriş görüntüsünün uzamsal çözünürlüğü konvolüsyon işlemiyle orijinal boyutun $1/4$ 'üne düşürülmektedir ($H/4 \times W/4$).

Aşama 1 (layer1). Birinci artık aşama, düşük seviyeli görsel özelliklerin çıkarıldığı katmandır:

- *BasicBlock* sayısı: 3,
- Çıkış kanal sayısı: 64,
- Tüm bloklarda stride 1 kullanılmıştır.

Bu aşamada uzamsal çözünürlük korunur ve kenar, köşe gibi temel görsel yapılar daha ayrıntılı biçimde temsil edilir.

Aşama 2 (layer2). İkinci aşamada kanal derinliği artırılırken uzamsal çözünürlük azaltılmaktadır:

- *BasicBlock* sayısı: 4,
- Çıkış kanal sayısı: 128,
- İlk blokta stride 2 kullanılarak çözünürlük $H/8 \times W/8$ seviyesine düşürülmüştür,
- Kalan üç blokta stride 1 kullanılmıştır.

Bu aşama, orta seviyeli semantik bilgilerin öğrenildiği katmandır ve alıcı alanın (receptive field) genişlemesine katkı sağlar.

Aşama 3 (layer3). Üçüncü aşama, daha soyut ve yüksek seviyeli özelliklerin çıkarıldığı katmandır:

- *BasicBlock* sayısı: 6,
- Çıkış kanal sayısı: 256,
- İlk blokta stride 2 kullanılarak çözünürlük $H/16 \times W/16$ seviyesine düşürülmüştür,
- Kalan beş blokta stride 1 kullanılmıştır.

Bu katmandaki özellik haritaları, nesne düzeyindeki semantik bilgileri temsil eder.

Aşama 4 (layer4). Encoder'ın son aşaması en soyut temsillerin elde edildiği katmandır:

- *BasicBlock* sayısı: 3,
- Çıkış kanal sayısı: 512,
- İlk blokta stride 2 kullanılarak çözünürlük $H/32 \times W/32$ seviyesine düşürülmüştür,
- Kalan iki blokta stride 1 kullanılmıştır.

Bu aşamada elde edilen özellik haritaları, yüksek semantik içerik ve geniş alıcı alan özelliklerine sahiptir.

Segmentasyon için Çok Ölçekli Özellik Çıktıları. Segmentasyon görevlerinde, encoder yapısındaki dört artık aşamanın (layer1–layer4) çıktıları farklı ölçeklerde özellik haritaları üretir. Bu çok ölçekli çıktılar, decoder yapısına atlama bağlantıları (skip connections) aracılığıyla aktarılmakta ve böylece hem uzamsal detayların hem de yüksek seviyeli semantik bilgilerin birlikte kullanılmasına olanak sağlamaktadır.

Modelin encoder kısmı, ImageNet üzerinde ön-eğitilmiş ResNet34 mimarisinden oluşur. Sınıflandırma katmanları çıkarılmış, ancak özellik çıkarıcı (feature extractor) katmanları korunmuştur. Encoder'ın her aşaması farklı çözünürlük ve kanal sayılarında özellik haritaları üretir:

$$E(x) = (e_1, e_2, e_3, e_4, e_5).$$

Bu özellikler:

- e_1 : yüksek çözünürlüklü, düşük seviyeli kenar/doku bilgisi,
- e_5 : düşük çözünürlüklü, yüksek seviyeli semantik bilgi

olmak üzere giderek soyutlaşan temsiller içerir.

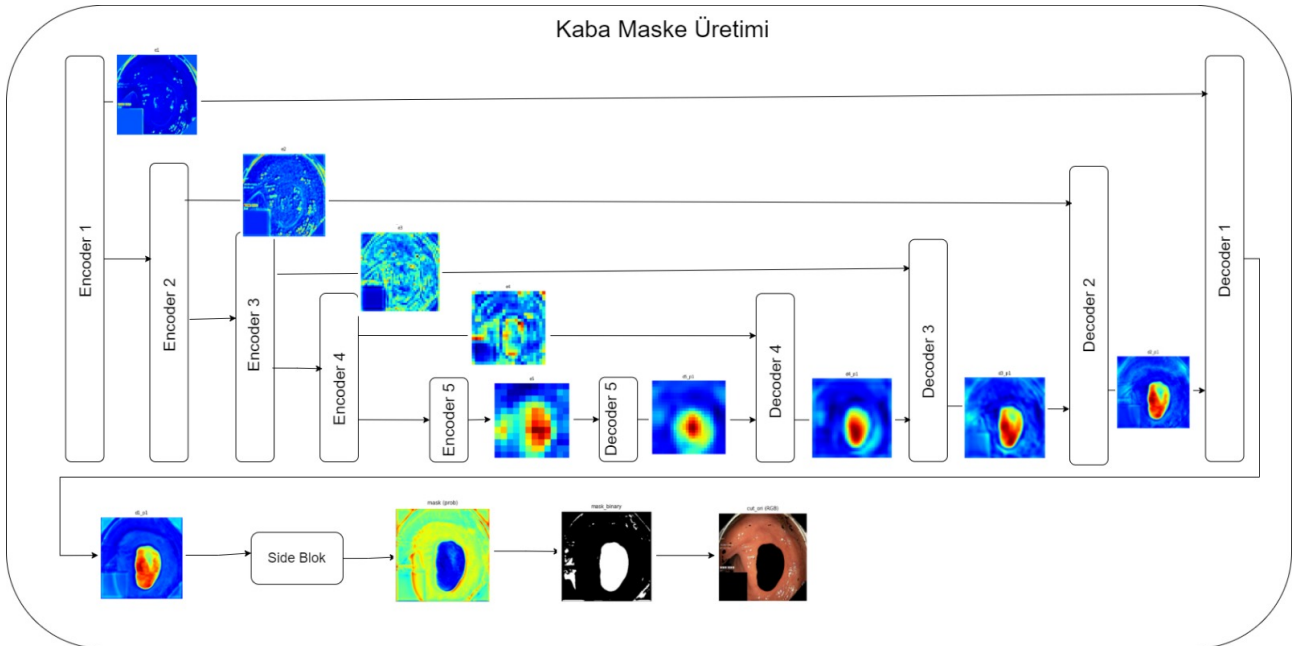


Figure 3: Kaba maske üretim diyagramı

3.5 U-Net Benzeri Segmentasyon Decoder

Segmentasyon decoder'ı, U-Net mimarisine benzer şekilde çok ölçekli upsampling ve atlama bağlantıları (skip connections) içeren bir yapıdadır. (bkz Fig.3) Decoder, encoder'dan elde edilen çok ölçekli özellik haritalarını kullanarak uzamsal çözünürlüğü kademeli olarak artırır ve piksel seviyesinde segmentasyon çıktısı üretir.

Her decoder aşamasında, bir önceki decoder çıktısı önce yukarı örneklenir (upsampling) ve ardından aynı çözünürlük seviyesindeki encoder çıktısı ile kanal boyutunda birleştirilir. Bu işlem matematiksel olarak aşağıdaki şekilde ifade edilir:

$$d_i = D^i \text{seg}([e_i, \text{Up}(d_{i+1})]), \quad i = 1, \dots, 4,$$

burada e_i encoder'ın i -inci aşamasından elde edilen özellik haritasını, $\text{Up}(\cdot)$ uzamsal çözünürlüğü iki katına çıkaran upsampling işlemini, $[\cdot, \cdot]$ ise kanal boyutunda birleştirmeyi göstermektedir.

Her D_{seg}^i bloğu, kanal sayısını azaltarak özellikleri rafine eden iki adet ardışık Conv-BN-ReLU katmanından oluşmaktadır. Bu yapı sayesinde, encoder'dan gelen yüksek çözünürlüklü uzamsal detaylar ile decoder'daki yüksek seviyeli semantik bilgiler etkili bir biçimde birleştirilmektedir.

Decoder'ın farklı aşamalarında ara segmentasyon çıktıları (side outputs) üretilerek derin gözetim (deep supervision) uygulanmaktadır. Bu ara çıktılar, eğitim sırasında kayıp fonksiyonuna dahil edilerek gradyan akışının güçlendirilmesi amaçlanmaktadır.

Son aşamada, en yüksek çözünürlüklü decoder çıktısı d_1 üzerinden bir 1×1 konvolüsyon ve sigmoid aktivasyonu uygulanarak segmentasyon olasılık haritası elde edilir:

$$\hat{Y}_{\text{coarse}} = D_{\text{seg}}(E_{\text{seg}}(x)) \quad (1)$$

$$\hat{Y} = \sigma(\text{Conv}_{1 \times 1}(d_1)), \quad \hat{Y} \in [0, 1]^{H \times W}.$$

4 Kayıp Fonksiyonları ve Öğrenme Tekniği

SemiMedSeg yarı denetimli bir öğrenme metodudur ve bu yöntemde sınırlı sayıdaki etiketli veriden maksimum verimi almak ve büyük miktardaki etiketsiz veriden yapısal özellikler öğrenmek amacıyla iki temel kayıp fonksiyonunun birleşimini kullanır. Modelin amacı, segmentasyon ve inpainting (görüntü tamamlama) eşzamanlı olarak optimize ederek ortak encoder (ResNet34) yapısını git gide daha yetenekli hale getirmektir.

4.1 Toplam Kayıp Fonksiyonu ($\mathcal{L}_{\text{total}}$)

Modelin öğrenme hedefi bahsedildiği üzere, denetimli öğrenme (segmentasyon) ve kendi kendini kontrol eden öğrenme (inpainting) hedeflerinin ağırlık toplamı olarak ifade edilir:

$$\mathcal{L} = \lambda_1 \sum_{x_l \in D_l} \mathcal{L}_{\text{seg}}(x_l) + \lambda_2 \sum_{x_u \in D_u} \mathcal{L}_{\text{inp}}(x_u) \quad (2)$$

Burada D_l etiketli veri kümesini, D_u ise etiketsiz veri kümesini temsil etmektedir. Makalede denge katsayıları $\lambda_1 = 2$ ve $\lambda_2 = 1$ olarak belirlenmiştir, yani segmentasyon görevi iki kat daha önceliklidir.

4.2 Kayıp Fonksiyonlarının Matematiksel Modellenmesi

4.2.1 Segmentasyon Kaybı ($\mathcal{L}_{seg}$)

Sadece etiketli veri üzerinden hesaplanan kayıptır ve modelin tıbbi lezyon bölgesini piksel hassasiyetinde ne kadar geniş ve doğru belirleyebildiğini ölçer. Doğruluğu arttırmak için iki farklı kayıp birleştirilmiştir:

- **Binary Cross-Entropy (BCE) Loss:** Piksel düzeyinde sınıflandırma yaparak her bir piksel için lezyon olma ($p = 1$), normal bölge olma ($p = 0$) olasılığına bakar.

$$\mathcal{L}_{BCE}^i = -\frac{1}{N} \sum_j y_{l,j}^i \log(\hat{y}_{l,j}^i) + (1 - y_{l,j}^i) \log(1 - \hat{y}_{l,j}^i) \quad (3)$$

- **Dice Loss:** Tahmin edilen maske ile gerçek (ground truth) maske arasındaki kesişim oranını ölçerek BCE kaybının zayıf kaldığı anlarda bölgesel olarak hata taraması yapar.

$$\mathcal{L}_{Dice}^i = 1 - \frac{2 \sum_j \hat{y}_{l,j}^i y_{l,j}^i + \epsilon}{\sum_j \hat{y}_{l,j}^i + \sum_j y_{l,j}^i + \epsilon} \quad (4)$$

Model, "Derin Denetim" (Deep Supervision) stratejisi kapsamında kaybı beş farklı dekode katmanının her birinde ayrı ayrı hesaplayarak toplar:

$$\mathcal{L}_{seg}(xl) = \sum_{i=1}^5 (\mathcal{L}_{BCE}^i(\hat{y}_l^i, y_l^i) + \mathcal{L}_{Dice}^i(\hat{y}_l^i, y_l^i)) \quad (5)$$

Burada i indisinin 1'den 5'e kadar gitmesinin nedeni, modelin temelini oluşturan ResNet34 kodlayıcısının (encoder) ve buna karşılık gelen U-Net kod çözücüsünün (decoder) 5 ana bloktan oluşmasıdır. Her katmanda ayrı kayıp hesaplanarak (Deep Supervision), gradyanların ağırlık en derin katmanlarına daha sağlıklı ulaşması ve her çözünürlük seviyesinde özelliklerin oluşturulması hedeflenir.

4.2.2 Inpainting Kaybı ($\mathcal{L}_{inp}$)

Etiketsiz verilerden bağlamsal bilgi (context information) ve yapısal özellikler çıkarmak için tasarlanan bu kayıp, maskelenmiş bölgelerin yeniden oluşturulmasına dayanır. İlk aşamada elde edilen kaba segmentasyon sonuçlarına göre orijinal görüntüdeki lezyon alanı silinir (x_{mask}).

Inpainting kaybı için L1 normu (mutlak hata) kullanılır. Bu hata, sadece maskelenmiş ($\bar{y}_u^i$) bölge içinde hesaplanmaktadır:

$$\mathcal{L}_{inp}(xu) = \sum_{i=1}^5 \bar{y}_u^i \times |\hat{x}_u^i - x_u^i| \quad (6)$$

Bu süreç, modelin lezyonların dokusal özelliklerini ve çevre dokuyla olan ilişkisini etiket yardımı olmaksızın öğrenmesini sağlar.

4.2.3 Gradyan Akışı ve Optimizasyon Mekanizması

Modelin eğitimi sırasında parametre güncellemeleri şu temel prensiplere dayanır:

- **Optimizer:** Ağırlık güncellemeleri için 0.9 momentum ve 0.00001 ağırlık azalımı (weight decay) ile Stochastic Gradient Descent algoritması kullanılır.

- **Öğrenme Oranı (Learning Rate):** Başlangıçta 0.001 olarak belirlenen öğrenme oranı, "Poly" stratejisiyle kademeli olarak düşürülür.
- **Ağırlık Güncelleme:** Hem $\mathcal{L}_{seg}$ hem de $\mathcal{L}_{inp}$ üzerinden gelen gradyanlar, Gated Feature Fusion (GFF) modülleri aracılığıyla ortak kodlayıcıya geri yayılır (backpropagation), böylece kodlayıcı her iki görev için de optimize edilir.

4.3 Optimizasyon ve Ağırlık Güncelleme Mekanizması

4.4 Gradyan

Sinir ağındaki her katman, girdi verisini $z = wx + b$ şeklinde bir doğrusal işleme tabi tutar. Amacımız, hatayı ($\mathcal{L}$) minimuma indirecek en uygun w (ağırlık) değerlerini bulmaktır. Türev (gradyan), hatanın hangi yönde arttığını gösterir. Biz hatayı azaltmak istediğimiz için gradyanın tam tersi yönünde hareket ederiz:

$$w_{t+1} = w_t - \eta \cdot \nabla_w \mathcal{L} \quad (7)$$

Burada η , öğrenme oranını temsil eder.

4.4.1 Zincir Kuralı (Chain Rule) ve w Ağırlıklarına Ulaşım

Hata çıktıda (output) hesaplanır, ancak asıl ağırlıklar ağın en başındaki "Shared Encoder" birimindedir. Bu derinlikteki ağırlıklara ulaşmak için Zincir Kuralı uygulanır:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \text{Tahmin}} \cdot \frac{\partial \text{Tahmin}}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (8)$$

$z = wx + b$ denkleminde $\frac{\partial z}{\partial w} = x$ olduğundan, bir ağırlığın ne kadar değişeceği doğrudan o katmana giren veriye (x) ve yukarıdan gelen hataya bağlıdır.

4.5 Değiştirilen Ağırlıkların Sisteme Etkisi

Ağırlıklar güncellendiğinde sistemde şu kritik değişiklikler gerçekleşir:

- **Özellik Seçiciliği:** Inpainting görevinden gelen gradyanlar modele dokuyu öğretirken; segmentasyon gradyanları bu bilgiyi sınır belirlemek için kullanır.
- **Kararlılık:** Poly öğrenme oranı politikası sayesinde, eğitim ilerledikçe ağırlık değişimleri küçülür, bu da modelin en iyi Dice skoruna (yaklaşık %87) ulaşmasını sağlar.
- **Sinerji:** Shared Encoder ağırlıkları, her iki kayıp fonksiyonunun ortak paydada bulunduğu bir noktaya çekilir; böylece etiketsiz veriden öğrenilen lezyon dokusu bilgisi segmentasyon başarısına doğrudan katkı sağlar.

5 Inpainting Süreci ve Gated Feature Fusion (GFF) Mekanizması

Bu bölümde, SemiMedSeg mimarisinde kullanılan *inpainting* (boşluk doldurma) süreci ve bu süreci segmentasyon ile ilişkilendiren *Gated Feature Fusion (GFF)* mekanizması, hem matematiksel hem de kod seviyesinde

ayrıntılı olarak açıklanmaktadır.

5.1 Kaba Segmentasyona Dayalı Maskeleme

Modelin ilk aşamasında, giriş görüntüsü $I \in \mathbb{R}^{H \times W \times 3}$ ortak encoder E_{seg} ve segmentasyon decoder'ı D_{seg} kullanılarak kaba bir segmentasyon çıktısı üretir:

$$\hat{Y}_{\text{coarse}} = D_{\text{seg}}(E_{\text{seg}}(x)) \quad (9)$$

Bu çıktı sigmoid fonksiyonu ile $[0, 1]$ aralığına sıkıştırılır ve ikili maske elde edilir:

$$\bar{Y}_{\text{coarse}} = \mathbb{I}(\hat{Y}_{\text{coarse}} > 0.5) \quad (10)$$

Kod karşılığı şu şekildedir:

```
mask = torch.sigmoid(mask)
mask_binary = (mask < 0.5).float()
mask_rbinary = (mask > 0.5).float()
```

Bu maske kullanılarak, lezyon bölgesi giriş görüntüsünden çıkarılır:

$$x_{\text{mask}} = x \odot (1 - \bar{Y}_{\text{coarse}}) \quad (11)$$

Kodda bu işlem:

```
cut_ori = ori * mask_rbinary
```

şeklinde gerçekleştirilir.

5.2 Inpainting için Encoder Özelliklerinin Çıkarılması

Maskeleme işleminden sonra, **ayrı bir encoder tanımlanmaz, orijinal görseller ve maskelenmiş görseller ağırlıkları aynı encoder'lerden geçtikten sonra orijinal görseller segmentasyon decoder'ına girerken maskelenmiş görseller inpainting decoder'ına girer.** Sonuç olarak aynı ResNet34 tabanlı encoder, hem orijinal görüntü x hem de maskelenmiş görüntü x_{mask} üzerinde çalıştırılır:

$$(e_1^{\text{seg}}, \dots, e_5^{\text{seg}}) = E(x) \quad (12)$$

$$(e_1^{\text{inp}}, \dots, e_5^{\text{inp}}) = E(x_{\text{mask}}) \quad (13)$$

Kod karşılığı:

```
inpe1, inpe2, inpe3, inpe4, inpe5 = self.encoder(cut_ori)
e1, e2, e3, e4, e5 = self.encoder(x)
```

Burada:

- e_i^{seg} : segmentasyon için çıkarılan özellikler
- e_i^{inp} : inpainting için çıkarılan özelliklerdir

5.3 Gated Feature Fusion (GFF) Mekanizması

Segmentasyon ve inpainting özellikleri doğrudan birleştirilmez. Bunun yerine, her ölçekte bir GFF modülü kullanılır.

i 'inci ölçekte GFF şu şekilde tanımlanır:

$$r_i = \sigma \left(W_r * [e_i^{\text{seg}}, e_i^{\text{inp}}] \right) \quad (14)$$

$$s_i = \sigma \left(W_s * [e_i^{\text{seg}}, e_i^{\text{inp}}] \right) \quad (15)$$

Burada:

- W_r ve W_s , 3×3 boyutunda konvülsyon filtreleridir ve 3×3 olmaları sayesinde sadece ilgili pikseli tekil değerlendirmek yerine etrafındaki pikselleri de göz önüne alarak inpainting ve segmentasyon özniteliklerinin ne seviyede işleme katılacağını ağırlıklandırılmasını sağlarlar.
- r_i : **reset gate** (inpainting bilgisinin ne kadar alınacağını belirler)
- s_i : **select gate** (hangi özelliğe güvenileceğini belirler)
- $\sigma(\cdot)$: sigmoid fonksiyonudur

Ara özellik hesaplanır:

$$\tilde{e}_i = W * (r_i \odot e_i^{\text{inp}}, e_i^{\text{seg}}) \quad (16)$$

Son birleşik özellik ise:

$$e_i = s_i \odot \tilde{e}_i + (1 - s_i) \odot e_i^{\text{seg}} \quad (17)$$

Kod karşılığı:

```
ge1 = self.gate1(e1, inpe1)
ge2 = self.gate2(e2, inpe2)
ge3 = self.gate3(e3, inpe3)
ge4 = self.gate4(e4, inpe4)
ge5 = self.gate5(e5, inpe5)
```

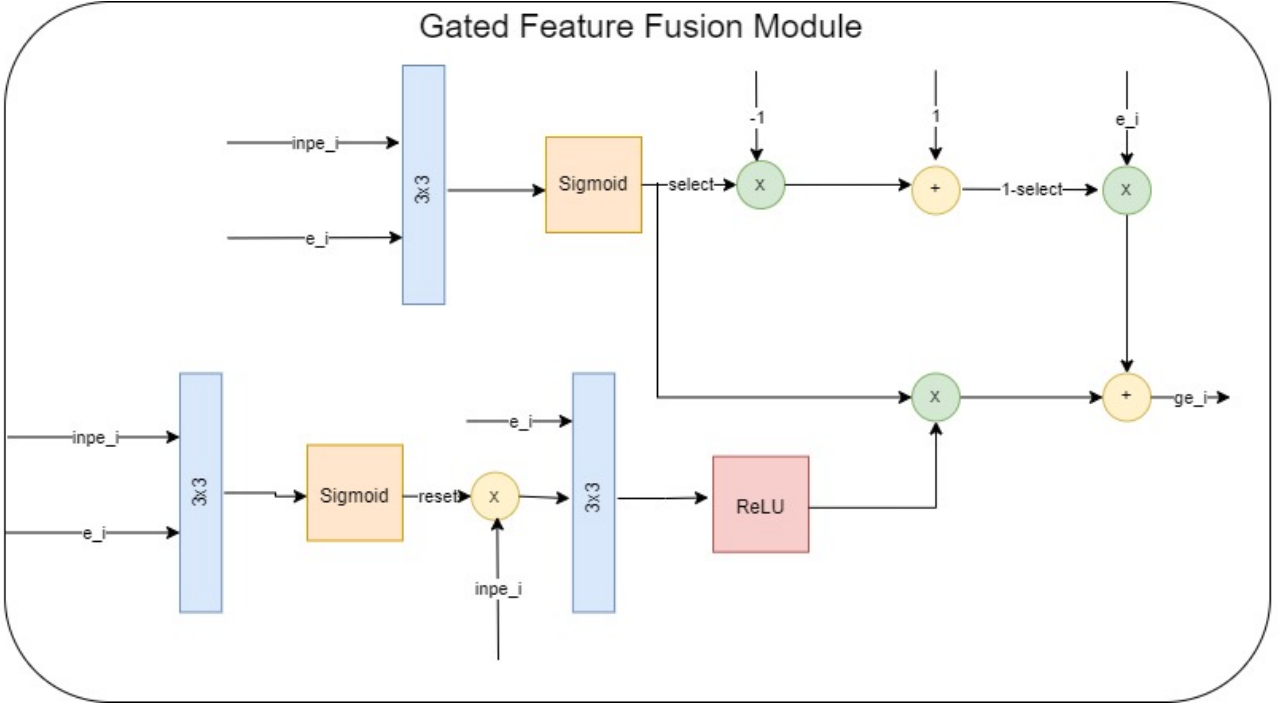


Figure 4: GFF Blok Diyagramı

5.4 Inpainting Decoder ile Boşluk Doldurma

GFF sonrası elde edilen e_i özellikleri, inpainting decoder'ına verilir:

$$\hat{x}^i = D_{\text{inp}}^i(e_i, d_{\text{inp}}^{i+1}) \quad (18)$$

Decoder blokları şu yapıyı izler:

- Transposed Convolution (upsampling)
- Batch Normalization
- ReLU aktivasyonu

Kodda bu süreç:

```
inpd5 = self.inpDecoder5(ge5)
inpd4 = self.inpDecoder4(cat(inpd5, ge4))
...
inpd1 = self.inpDecoder1(cat(inpd2, ge1))
```

Son inpainting çıktısı:

$$\hat{I}_{\text{inp}} = \hat{x} \odot \bar{Y}_{\text{coarse}} + x \odot (1 - \bar{Y}_{\text{coarse}}) \quad (19)$$

Kodda:

```
inping = inping * mask_binary + ori * mask_rbinary
```

5.5 Coarse-to-Fine Segmentasyon ile Nihai Maske Üretimi

Inpainting decoder çıktısı ile maskelenmiş bölgeler doldurulduktan sonra, ağ segmentasyon görevini *coarse-to-fine* mantığı ile yeniden ele alır. Bu aşamada amaç, ilk kaba segmentasyon sonucundaki hataları (GFF ve inpainting sayesinde zenginleşmiş öznitelikleri kullanarak) düzeltmektir.

GFF modüllerinden elde edilen birleşik özellikler g_i , segmentasyon decoder'ına tekrar beslenir:

$$\hat{y}_{\text{fine}}^i = \begin{cases} D_{\text{seg}}^i(g_i, d_{\text{seg}}^{i+1}), & i = 1, 2, 3, 4 \\ D_{\text{seg}}^5(g_5), & i = 5 \end{cases} \quad (20)$$

Burada D_{seg}^i ilgili segmentasyon decoder katmanını, d_{seg}^{i+1} ise bir üst decoder katmanından gelen özellikleri ifade eder. Bu yapı klasik U-Net benzeri skip-connection mantığını korurken, GFF ile filtrelenmiş ve düzeltilmiş özniteliklerin kullanılmasını sağlar.

Kodda bu süreç aşağıdaki şekilde gerçekleştirilir:

```
d5 = self.segDecoder5(ge5)
sidemask5 = self.segSideout5(d5)

d4 = self.segDecoder4(cat(d5, ge4))
sidemask4 = self.segSideout4(d4)

d3 = self.segDecoder3(cat(d4, ge3))
sidemask3 = self.segSideout3(d3)

d2 = self.segDecoder2(cat(d3, ge2))
sidemask2 = self.segSideout2(d2)

d1 = self.segDecoder1(cat(d2, ge1))
mask = self.segconv(d1)
mask = torch.sigmoid(mask)
```

Ara katmanlardan üretilen `sidemask` çıktıları, *deep supervision* amacıyla kullanılır ve segmentasyon decoder'ının her seviyede anlamlı özellikler öğrenmesini sağlar.

Son olarak sigmoid aktivasyonu ile elde edilen çıktı:

$$\hat{Y} = \sigma(D_{\text{seg}}(g_1)) \quad (21)$$

ağın nihai segmentasyon tahmini olup, ilk kaba maske $\hat{Y}_{\text{coarse}}$ 'a kıyasla daha keskin sınırlar ve daha doğru lezyon bölgeleri içerir.

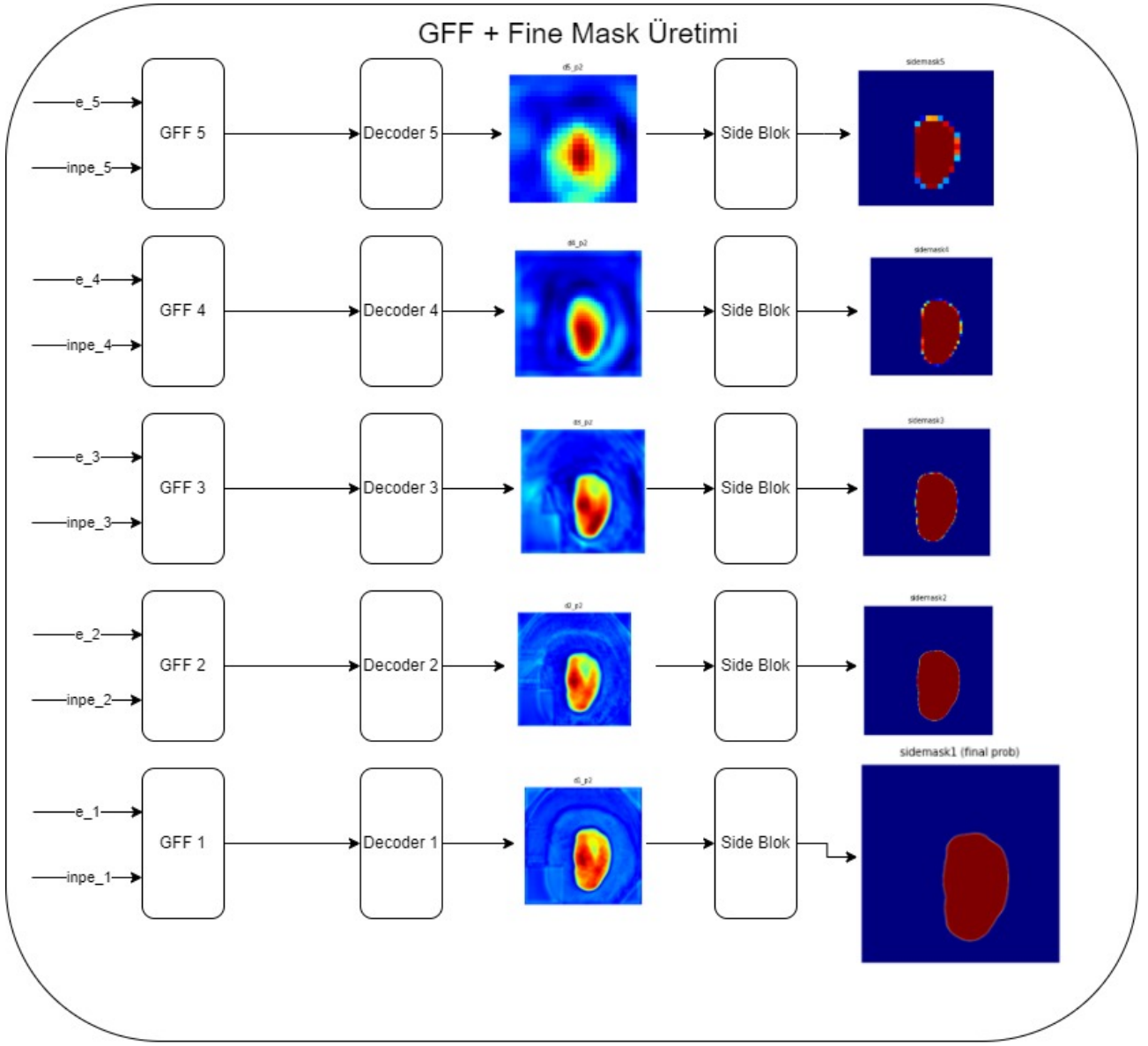


Figure 5: Fine Mask Blok Diyagramı

5.6 Öğrenme Mantığı ve Düzeltme (Correction Learning)

Eğer segmentasyon hatalıysa, maskeleme de hatalı olur. Bu durumda inpainting ağı, yanlış bölgeleri doldurmak zorunda kalır ve inpainting kaybı yükselir.

Bu hata, GFF üzerinden segmentasyon özelliklerine geri yansıtılır. Böylece model:

“Eğer segmentasyon doğru olsaydı, inpainting daha kolay olurdu”

mantığıyla kendini düzeltir.

Bu mekanizma **self-supervised correction learning** olarak adlandırılır ve özellikle etiketlenmemiş verilerde segmentasyon performansını dolaylı yoldan artırır.

6 Deneysel Sonuçlar ve Detaylı Analiz

Kvasir-SEG veri kümesi üzerinde gerçekleştirilen yarı denetimli (semi-supervised) eğitim sürecinin tamamlanmasının ardından, modelin başarımı eğitimden tamamen bağımsız tutulan test kümesi üzerinde değer-

lendirilmiştir. Bu bölümde, notebook *FinalTest.ipynb* çıktılarından elde edilen nicel (sayısal) ve nitel (görsel) analiz sonuçları sunulmaktadır.

6.1 Test Kümesi Performansı (Nicel Analiz)

Eğitim sürecinde validasyon kümesi üzerinde en iyi performansı sağlayan model ağırlıkları (checkpoint) kullanılarak, 200 görüntüden oluşan test kümesi üzerinde çıkarım (inference) işlemi gerçekleştirilmiştir.

Yarı denetimli eğitim sürecinde kullanılan Kvasir-SEG eğitim konfigürasyonu Tablo 1’de özetlenmiştir. Bu konfigürasyonda, modelin omurga (backbone) ağırlıkları ImageNet veri kümesi üzerinde önceden eğitilmiş ResNet34 parametrelerinden başlatılmıştır. Eğitim, başlangıç öğrenme oranı 1×10^{-4} olacak şekilde toplam 80 epoch boyunca yürütülmüş; etiketli ve etiketsiz veri yükleyicileri için batch boyutu ayrı ayrı 4 olarak belirlenmiştir. Eğitim sürecinde 120 adet etiketli (%20) ve 480 adet etiketsiz (%80) görüntü kullanılmıştır.

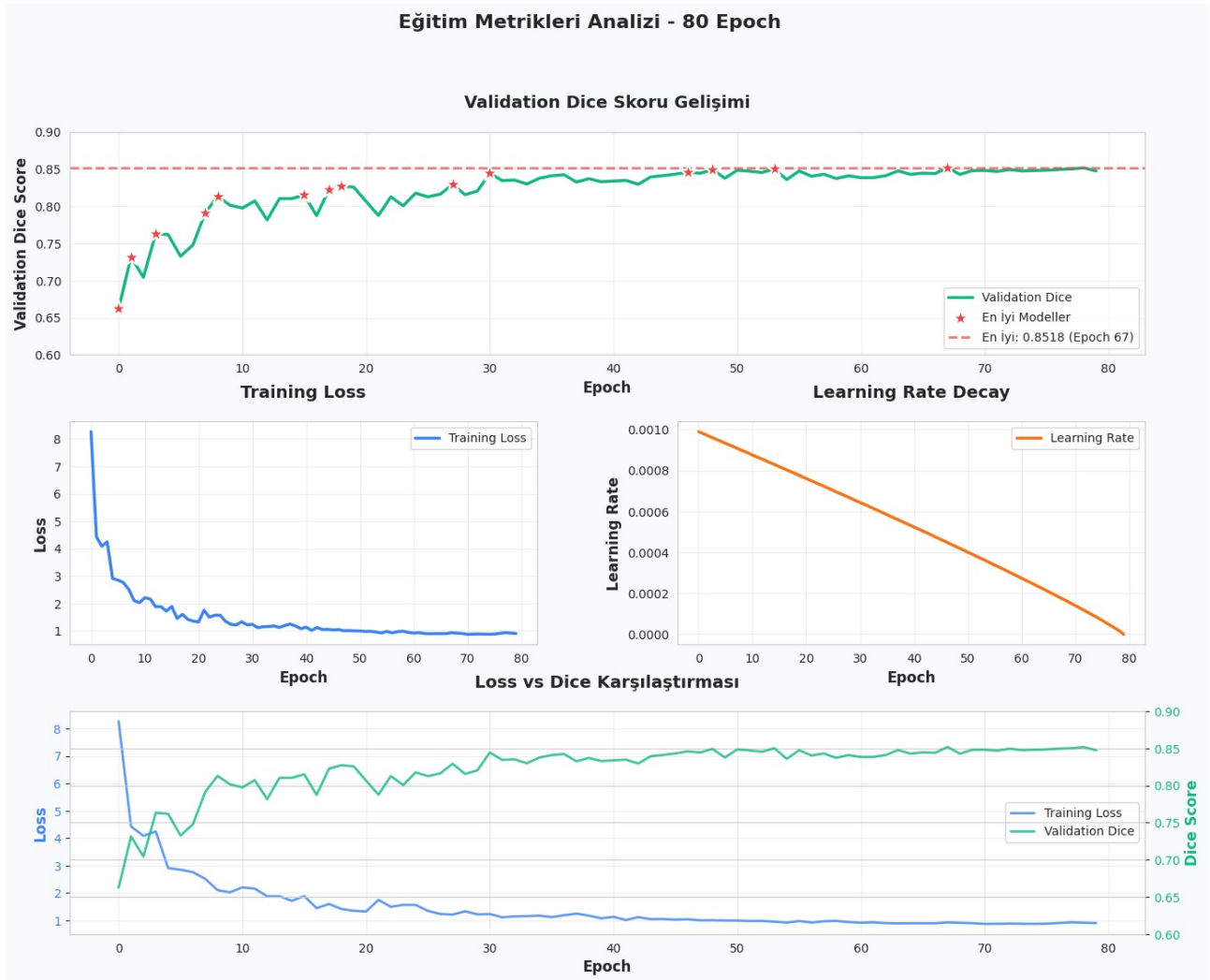


Figure 6: Eğitim Metrikleri

6.2 Kvasir-SEG Test Kümesi Üzerinde Elde Edilen Performans Sonuçları

Önerilen modelin Kvasir-SEG test kümesi üzerindeki performansı, çeşitli piksel tabanlı değerlendirme metrikleri kullanılarak analiz edilmiştir. Elde edilen sonuçlar Tablo 2’de sunulmaktadır.

Table 1: Kvasir-SEG Yarı Denetimli Eğitim Konfigürasyonu

Parametre	Değer
Başlangıç Ağırlıkları	ImageNet Pretrained (ResNet34)
Öğrenme Oranı (Learning Rate)	0.0001
Epoch Sayısı	80
Batch Boyutu	4 (Etiketli) + 4 (Etiketsiz)
Etiketli Eğitim Verisi	120 görüntü (%20)
Etiketsiz Eğitim Verisi	480 görüntü (%80)
Test Verisi	200 görüntü

Model, **0.9578 doğruluk (Accuracy)** değeri ile piksellerin %95'inden fazlasını doğru sınıflandırmıştır. **0.9823 Specificity** değeri, yanlış pozitif oranının oldukça düşük olduğunu ve sağlıklı dokuların polip olarak işaretlenmediğini göstermektedir. **0.8777 Recall** değeri, polip bölgelerinin büyük bir kısmının başarıyla tespit edildiğini ortaya koymaktadır.

0.8689 Precision değeri, modelin polip olarak tahmin ettiği bölgelerin yüksek güvenilirliğe sahip olduğunu göstermektedir. Precision ve Recall arasındaki dengeyi ifade eden **F1-skoru 0.8533** olarak hesaplanmıştır. Tıbbi görüntüleme uygulamalarında gözden kaçırmanın maliyeti yüksek olduğundan, Recall'a daha fazla ağırlık veren **F2-skorunun 0.8609** seviyesinde olması model açısından önemli bir avantaj sağlamaktadır.

Segmentasyon başarısını ölçen **IoU (Polyp)** metriği 0.7788 olarak elde edilirken, **IoU (Background)** değeri 0.9470 ile arka plan segmentasyonunun neredeyse kusursuz olduğunu göstermektedir.

Elde edilen **0.8533 F1-skoru** ve **0.7788 IoU (Polyp)** değerleri, modelin yalnızca %20 etiketli veri kullanmasına rağmen tam denetimli (fully-supervised) yöntemlere yakın, rekabetçi bir performans sergilediğini göstermektedir. Özellikle yüksek Specificity değeri, modelin klinik uygulamalarda yanlış alarmları minimize etme potansiyeline sahip olduğunu ortaya koymaktadır.

Table 2: Kvasir-SEG Test Kümesi Üzerinde Elde Edilen Performans Sonuçları

Metrik	Skor	Yorum
Accuracy (Doğruluk)	0.9578	Model, piksellerin %95'inden fazlasını doğru sınıflandırmıştır.
Specificity (Seçicilik)	0.9823	Yanlış pozitif oranı oldukça düşüktür; sağlıklı dokular polip olarak işaretlenmemektedir.
Recall (Duyarlılık)	0.8777	Polip bölgelerinin büyük bir kısmı başarıyla tespit edilmiştir.
Precision (Kesinlik)	0.8689	Polip olarak tahmin edilen bölgelerin güvenilirliği yüksektir.
F1-Skoru	0.8533	Precision ve Recall arasındaki dengeyi gösteren temel başarı ölçütüdür.
F2-Skoru	0.8609	Recall'a daha fazla ağırlık veren ve tıbbi teşhis açısından kritik bir metriktir.
IoU (Polyp)	0.7788	Tahmin edilen maske ile gerçek maske arasındaki örtüşme başarısını göstermektedir.
IoU (Background)	0.9470	Arka plan segmentasyonu neredeyse kusursuzdur.

6.3 Tartışma

Deneysel sonuçlar, önerilen Self-Supervised Correction Learning yapısının, etiketlenmemiş veriyi (eğitim setinin %80'i) etkin bir şekilde kullandığını doğrulamaktadır. Sadece 120 etiketli görüntü ile eğitilen bir modelin, test

setinde 0.85 F1 skorunu aşması, inpainting yardımcı görevinin (auxiliary task) segmentasyon ana görevine pozitif bir regülasyon sağladığını göstermektedir. Geleneksel yöntemlerde az veri ile eğitim yapıldığında görülen aşırı öğrenme (overfitting) problemi, bu modelde inpainting kaybının getirdiği kısıtlamalar ve güçlü veri artırma (augmentation) teknikleri sayesinde minimize edilmiştir.

6.4 Transfer Öğrenme ve Genelleştirme Analizi (ISBI 2016)

Derin öğrenme modellerinin en büyük zorluklarından biri, eğitim verisinden farklı karakteristiğe sahip yeni veriler üzerinde performansını koruyabilmesidir (Generalization). Bu çalışmada, Kvasir-SEG veri setinde eğitilen modelin ağırlıkları kullanılarak, farklı bir endoskopik veri seti olan ISBI 2016 üzerinde Transfer Öğrenme (Transfer Learning) deneyi gerçekleştirilmiştir.

6.5 Deney Kurgusu ve Hiperparametreler

Model, "sıfırdan" (scratch) eğitilmek yerine, Kvasir-SEG üzerinde öğrenilen özniteliklerle başlatılmış (pretrained weights) ve ISBI 2016 veri setine uyarlanmıştır (fine-tuning). Bu yaklaşım, veri setleri arasındaki yapısal benzerliklerden faydalanarak eğitimi hızlandırmayı ve az veriyle daha iyi sonuç almayı hedefler.

Kullanılan ISBI 2016 veri kümesi için transfer öğrenme eğitim konfigürasyonu Tablo 3'da sunulmuştur. Modelin başlangıç ağırlıkları, Kvasir-SEG veri kümesi üzerinde önceden eğitilmiş (pretrained) parametrelerden elde edilmiştir. Eğitim sürecinde öğrenme oranı 5×10^{-4} olarak belirlenmiş ve toplam 60 epoch boyunca eğitim gerçekleştirilmiştir. Batch boyutu 4 olarak seçilmiştir. Eğitim aşamasında 144 adet etiketli ve 576 adet etiketsiz görüntü kullanılmıştır. Model performansının değerlendirilmesi ise 379 adet test görüntüsü üzerinden yapılmıştır.

Table 3: ISBI 2016 Transfer Öğrenme Eğitim Konfigürasyonu

Parametre	Değer
Başlangıç Ağırlıkları	Kvasir-SEG Pretrained
Öğrenme Oranı (Learning Rate)	0.0005
Epoch Sayısı	60
Batch Boyutu	4
Etiketli Eğitim Verisi	144 görüntü
Etiketsiz Eğitim Verisi	576 görüntü
Test Verisi	379 görüntü

6.6 Train Sonuçları

Eğitim tamamlandıktan sonra 379 görüntüden oluşan ISBI 2016 test kümesi üzerinde model değerlendirilmiştir. Elde edilen nicel sonuçlar aşağıdaki tabloda sunulmuştur.

Modelin ISBI 2016 test kümesi üzerindeki performansı, piksel tabanlı değerlendirme metrikleri kullanılarak incelenmiştir. Elde edilen nicel sonuçlar Tablo 4'de sunulmaktadır.

Model, **0.7652 Accuracy** değeri ile genel piksel sınıflandırmasında orta seviyede bir başarı sergilemiştir. **0.8719 Specificity** değeri, modelin arka planı ayırt etme konusunda görece başarılı olduğunu ve yanlış poz-

itiflerin sınırlı kaldığını göstermektedir. Buna karşın, **0.6335 Recall** değeri, polip bölgelerinin tespit edilme oranının Kvasir-SEG sonuçlarına kıyasla belirgin şekilde düştüğünü ortaya koymaktadır.

0.5894 Precision değeri, modelin polip olarak tahmin ettiği bölgelerdeki doğruluk oranının sınırlı kaldığını göstermektedir. Precision ve Recall arasındaki dengeyi ifade eden **F1-skoru 0.5249** olarak hesaplanmış olup, Kvasir-SEG veri setinde elde edilen yaklaşık **0.80 F1** değerine kıyasla anlamlı bir performans düşüşü gözlenmiştir.

Segmentasyon başarısını ölçen **IoU (Polyp)** metriği **0.4075** olarak elde edilirken, **IoU (Mean)** değeri **0.5651** seviyesindedir. Bu sonuçlar, tahmin edilen polip maskelerinin yer gerçeği ile örtüşme oranının ISBI 2016 veri setinde belirgin şekilde azaldığını göstermektedir

Table 4: ISBI 2016 Test Kümesi Üzerinde Elde Edilen Performans Sonuçları

Metrik	Skor	Analiz
Accuracy	0.7652	Genel piksel sınıflandırma başarısını göstermektedir.
Specificity	0.8719	Model arka plan sınıfını ayırt etmekte görece başarılıdır.
Recall	0.6335	Polip tespit oranı Kvasir-SEG veri setine kıyasla düşüktür.
Precision	0.5894	Tespit edilen polip bölgelerindeki doğruluk oranıdır.
F1-Skoru	0.5249	Kvasir-SEG sonuçlarına (yaklaşık 0.80) kıyasla belirgin düşüş göstermektedir.
IoU (Polyp)	0.4075	Tahmin edilen polip maskeleri ile yer gerçeği arasındaki örtüşme oranıdır.
IoU (Mean)	0.5651	Sınıflar arası ortalama örtüşme başarısını göstermektedir.

Eğitilmiş Model Linki

7 Referanslar

- [1] R. Zhang, S. Liu, Y. Yu, G. Li, “Self-Supervised Correction Learning for Semi-Supervised Biomedical Image Segmentation”, *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2021.
- [2] ReaFly, *SemiMedSeg GitHub deposu*, <https://github.com/ReaFly/SemiMedSeg>, erişim tarihi: December 24, 2025.
- [3] Meta AI online publish, “Self-supervised learning: The dark matter of intelligence” 2021
- [4] D. Jha, P. H. Smedsrud, M. A. Riegler, P. Halvorsen, T. de Lange, D. Johansen, H. D. Johansen, “Kvasir-SEG: A segmented polyp dataset”, *International Conference on Multimedia Modeling (MMM)*, pp. 451–462, Springer, 2020.
- [5] D. Gutman, N. C. Codella, E. Celebi, B. Helba, M. Marchetti, N. Mishra, A. Halpern, “Skin lesion analysis toward melanoma detection: A challenge at the International Symposium on Biomedical Imaging (ISBI) 2016, hosted by the International Skin Imaging Collaboration (ISIC)”, *arXiv preprint arXiv:1605.01397*, 2016.
- [6] O. Ronneberger, P. Fischer, T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation”, *arXiv preprint arXiv:1505.04597*, 2015.
- [7] K. He, X. Zhang, S. Ren, J. Sun, “Deep Residual Learning for Image Recognition”, *arXiv preprint arXiv:1512.03385*, 2015.
- [8] Alpaydın, E. “Yapay öğrenme”, *Bogazici Universitesi Yayinevi*, 2011.